Using Android Preferences

Tanusree Nath
PhD Research Scholar
Department of Computer Science, BHU

Google scholar: https://scholar.google.com/citations?user=h80-tYYAAAAJ&hl=en
LinkedIn: https://www.linkedin.com/in/tanusree-nath-5447291a7/

Android Preferences – Definition

- Android Preferences are used to store **simple persistent data**.
- Data is stored in the form of **key–value pairs**.
- The stored data remains available even after the app is closed.

Why use preferences?

- To save **user-specific settings and configurations**
- Lightweight storage (not suitable for large/complex data)
- Easy to implement and access

Use Cases of preferences

- **Dark mode toggle** - This is the most common use case. It allows users to switch between Light, Dark, or System Default themes.
- **Language Selection** - let users override the system language for just your app.
- **Remember user login** - While you should **not** store passwords here, you can store a boolean to remember if the user opted to stay logged in, or store their preferred login method.
- **Notification settings** - Granular control over notifications is a standard expectation for mobile users.

SharedPreferences

- **SharedPreferences** is the Android API used to implement preferences. Data remains stored even after app is closed/device restarted.
- It allows storing and retrieving primitive data types.

Supported data types:

❖int

❖float

❖boolean

❖long

❖string

Key Methods

Step	Method	Purpose
1	getSharedPreferences(name, mode)	Obtain SharedPreferences instance
2	edit()	Begin a write operation
3	putString(), putInt(), putBoolean(), etc.	Add/modify key-value pairs
4	apply() or commit()	Save changes

Storing data in SharedPreferences

- Data is stored internally as an XML file
- Stored using key-value pairs
- Each app has its own private preferences file (by default)

Example:

```
// 1. Get SharedPreferences instance
val sharedPref = getSharedPreferences("UserSettings", Context.MODE_PRIVATE)

// 2. Edit and save data
sharedPref.edit()
    .putString("username", "AlexJohnson")
    .putInt("highScore", 2500)
    .putBoolean("soundEnabled", true)
    .putFloat("volume", 0.75f)
    .putLong("lastPlayTime", System.currentTimeMillis())
    .apply()  // Async save
```

Content of UserSettings.xml

```xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="username">AlexJohnson</string>
    <int name="highScore" value="2500" />
    <boolean name="soundEnabled" value="true" />
    <float name="volume" value="0.75" />
    <long name="lastPlayTime" value="1702345678901" />
</map>
```

Complete working example - Storing

```
class MainActivity : AppCompatActivity() {

    // Save data
    fun saveUserPreferences() {
        val prefs = getSharedPreferences("MyAppPrefs", MODE_PRIVATE)

        prefs.edit().apply {
            putString("userName", "Sarah")
            putInt("age", 28)
            putBoolean("notifications", true)
            commit()  // or apply()
        }
```

Complete working example (contd.) - Retrieval

```
    // Read data back
    fun loadUserPreferences() {
        val prefs = getSharedPreferences("MyAppPrefs", MODE_PRIVATE)

        val userName = prefs.getString("userName", "Default Name")
        val age = prefs.getInt("age", 0)
        val notifications = prefs.getBoolean("notifications", false)

        println("Loaded: $userName, $age years old, Notifications: $notifications")
    }
}
```

Complete working example (contd.) - Removal

```
fun exampleRemoval() {
    val prefs = getSharedPreferences("MyAppPrefs", MODE_PRIVATE)

    // Remove a single key-value pair
    prefs.edit().remove("userName").apply()

    // Remove multiple key-value pairs (list)
    prefs.edit().apply {
        remove("age")
        remove("notifications")
        apply()
    }
}
```

Preference UI (User Settings Screen)

- Preference UI provides a default layout where defined preferences are automatically converted into UI components and displayed, without manually designing the interface.
- It allows users to **view and modify preferences** easily without designing custom layouts.

Architecture overview:

PreferenceFragmentCompat

↓

PreferenceScreen (XML) – Root container of all preferences

↓

PreferenceCategory (optional) – Groups related settings

↓

Preference Components (Switch, List, etc.)

↓

Stored in SharedPreferences (XML file)

Preference UI – PreferenceFragmentCompat

PreferenceFragmentCompat (The Manager) - "Brain" of the operation. It is a special Fragment that hosts your settings UI. You don't design the layout here; you just tell it which XML file to load.

```
class MySettingsFragment : PreferenceFragmentCompat() {
    override fun onCreatePreferences(savedInstanceState: Bundle?, rootKey: String?) {
        // Step 1: Point to your XML file
        setPreferencesFromResource(R.xml.my_prefs, rootKey)
    }
}
```

Preference UI – PreferenceScreen

- The “Container” of the operation.
- This is the root tag in your XML file. It holds everything else.

```
<PreferenceScreen xmlns:app="http://schemas.android.com/apk/res-auto">

    <!-- add contents here -->

</PreferenceScreen>
```

PreferenceUI - PreferenceCategory

- The “Organizer” of the operation.
- This is optional but helpful. It adds a visual header (like "Account" or "Privacy") to group related settings together, making the list easier to read.

```
<PreferenceCategory app:title="Notifications">

    <!-- related settings go here -->

</PreferenceCategory>
```

Preference UI – Preference Components

- The “Interactive Bits” of the operation.
- These are the actual UI elements the user touches. Each one must have a key—this is the "ID" used to save the data.
  - **Switch:** For On/Off toggles.

    Use SwitchPreferenceCompat
  - **List:** For picking one option from many.

    Use ListPreference

```
<SwitchPreferenceCompat
    app:key="notif_enabled"
    app:title="Enable Push Notifications"
    app:defaultValue="true" />

<ListPreference
    app:key="font_size"
    app:title="Font Size"
    app:entries="@array/font_names"
    app:entryValues="@array/font_values" />
```

Preference UI – Reflecting in SharedPreferences

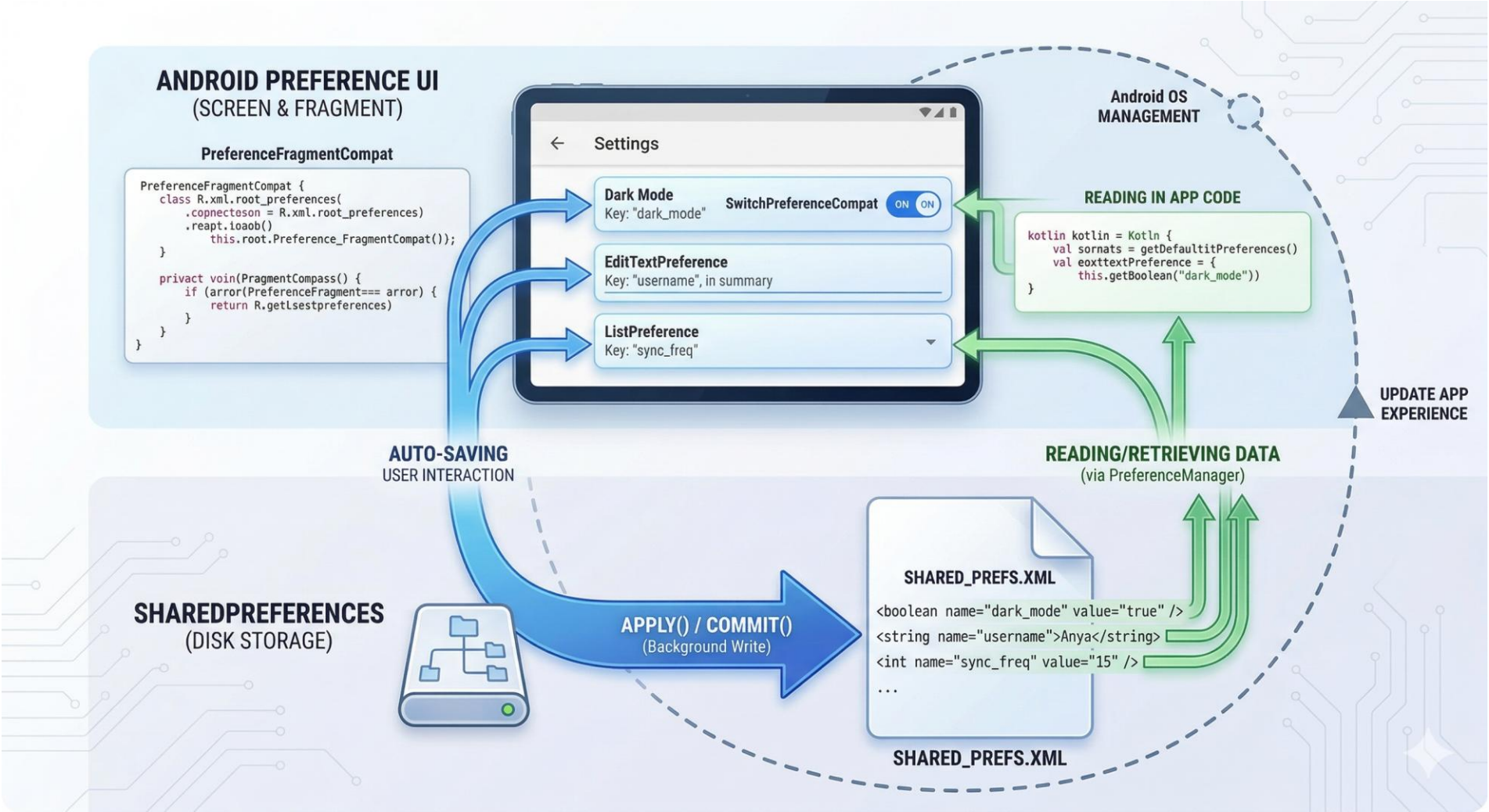
- The magic of this pipeline is that you **never** have to write code to save the data. When the user flips a switch, the library automatically saves it to SharedPreferences.
- To use that data later in your app (like in an Activity), you just "pull" it out using the key.

```
val sharedPrefs = PreferenceManager.getDefaultSharedPreferences(context)

// Read the data using the 'key' defined in XML
val isEnabled = sharedPrefs.getBoolean("notif_enabled", true)
val fontSize = sharedPrefs.getString("font_size", "medium")

println("User wants notifications: $isEnabled")
```

Preference UI- Complete XML code

```xml
<PreferenceScreen xmlns:app="http://schemas.android.com/apk/res-auto">

    <PreferenceCategory app:title="General Settings">

        <SwitchPreferenceCompat
            app:key="sync_data"
            app:title="Sync Data"
            app:summary="Auto-sync in background"
            app:defaultValue="true" />

        <EditTextPreference
            app:key="user_nickname"
            app:title="Nickname"
            app:useSimpleSummaryProvider="true" />

    </PreferenceCategory>

</PreferenceScreen>
```

The total workflow



Security Considerations

When building settings, it’s easy to forget that SharedPreferences (the default storage for the Preference Library) saves data in a **plain-text XML file** on the device. If a device is rooted or compromised, this data is easily readable.

You should never store the following in plain text:

- **Passwords:** Never store raw passwords.
- **Sensitive Tokens:** API keys, OAuth tokens, or Session IDs.
- **Personally Identifiable Information (PII):** Credit card numbers, social security numbers, or health data.

The Secure Solution: EncryptedSharedPreferences

If you need to store something semi-sensitive (like a user's email or a non-critical access token), use the **Security library**. It encrypts both the **keys** and the **values**.

```
dependencies {
    implementation("androidx.security:security-crypto-ktx:1.1.0-alpha06")
}
```

Best practices – apply() vs commit()

- When you manually save data to SharedPreferences, you have two choices. **Always prefer apply()**.
  - ❑**apply():** Saves the data to memory immediately and writes to the disk in the **background**. It doesn't block your app's UI.
  - ❑**commit():** Writes to the disk **synchronously** and returns a boolean. This can freeze your app if the file is large.

```
// DO THIS (Fast & Safe)
prefs.edit().putBoolean("key", true).apply()

// AVOID THIS (Slows down UI)
prefs.edit().putBoolean("key", true).commit()
```

Best Practices (contd.) - Use Meaningful Keys

- Keys are the "ID" for your data. If you change a key in a later app update, the user's saved setting will be lost because the app can't find it anymore.
- **Bad Key:** "checkbox1" (What does it do?)
- **Good Key:** "pref_enable_dark_mode"

Best Practices (contd.) – Avoid large data

- SharedPreferences is a simple XML file. Every time you read a single value, Android often has to load the **entire file** into memory.
- **Use it for:** Booleans, small strings, and simple integers (Settings).
- **Don't use it for:** JSON strings, large lists of objects, or images.
- **Alternative:** For complex data, use a **Room Database**.

Thank You